

## 3.1 Guía de Funciones: KNN y Naive Bayes en R y Python

### R — class + e1071

```
library(class) # KNN
library(e1071) # Naive Bayes
library(caret) # evaluación y CV

data(iris)
set.seed(42)

# PREPROCESAMIENTO
# -----
# División train/test (80/20)
idx <- createDataPartition(iris$Species, p = 0.8, list = FALSE)
train <- iris[idx, ]
test <- iris[-idx, ]

# Normalización (NECESARIA para KNN)
preproc <- preProcess(train[, 1:4], method = c("center", "scale"))
train_norm <- predict(preproc, train[, 1:4])
test_norm <- predict(preproc, test[, 1:4])
# preProcess: calcula media y SD del train
# predict(preproc, ...): aplica la misma transformación al test

# KNN
# -----
pred_knn <- knn(
  train = train_norm, # datos de entrenamiento (features)
  test = test_norm, # datos de test (features)
  cl = train$Species, # etiquetas de entrenamiento
  k = 5 # número de vecinos
)
```

```
# knn() retorna factor con predicciones

# Evaluar KNN
conf_knn <- confusionMatrix(pred_knn, test$Species)
print(conf_knn)
# → Accuracy, Kappa, Sensitivity, Specificity por clase

# Encontrar K óptimo
k_vals <- 1:20
errores <- sapply(k_vals, function(k) {
  pred <- knn(train_norm, test_norm, train$Species, k=k)
  mean(pred != test$Species) # tasa de error
})
plot(k_vals, errores, type="b",
     xlab="K", ylab="Tasa de error",
     main="KNN: selección de K")
k_optimo <- k_vals[which.min(errores)]
cat("K óptimo:", k_optimo, "
")

# NAIVE BAYES
# -----
modelo_nb <- naiveBayes(Species ~ ., data = train)
# Fórmula: variable_objetivo ~ . (punto = todas las features)

pred_nb <- predict(modelo_nb, test)
# predict retorna factor con clases predichas

conf_nb <- confusionMatrix(pred_nb, test$Species)
print(conf_nb)

# Ver probabilidades a posteriori
probs_nb <- predict(modelo_nb, test, type = "raw")
# type="raw": retorna matriz de probabilidades por clase
head(probs_nb)
```

## Python — sklearn

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

X, y = load_iris(return_X_y=True)
target_names = load_iris().target_names

# División train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# stratify=y: mantiene proporción de clases en cada split

# Normalización
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train) # fit+transform en train
X_test_s = scaler.transform(X_test)      # solo transform en test (!)

# KNN
knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(X_train_s, y_train)
y_pred_knn = knn.predict(X_test_s)

print("KNN - Reporte de clasificación:")
print(classification_report(y_test, y_pred_knn, target_names=target_names))

# Matriz de confusión
cm = confusion_matrix(y_test, y_pred_knn)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=target_names, yticklabels=target_names)
plt.title('Matriz de confusión - KNN')
plt.ylabel('Real'); plt.xlabel('Predicho')
plt.tight_layout()

# Selección de K óptimo
k_range = range(1, 21)
scores = [cross_val_score(KNeighborsClassifier(n_neighbors=k),
                          X_train_s, y_train, cv=5).mean()
          for k in k_range]
k_optimo = k_range[np.argmax(scores)]
print(f"K óptimo: {k_optimo} (accuracy CV: {max(scores):.3f})")

# NAIVE BAYES
nb = GaussianNB()
nb.fit(X_train, y_train) # NB no necesita normalización
```

```
y_pred_nb = nb.predict(X_test)
print("
Naive Bayes – Reporte:")
print(classification_report(y_test, y_pred_nb, target_names=target_names))

# Probabilidades
probs = nb.predict_proba(X_test[:5])
for i, prob in enumerate(probs):
    print(f" Muestra {i+1}: {dict(zip(target_names, prob.round(3)))}")
```